

# FULL STACK DEVELOPMENT WITH MERN

## PROJECT DOCUMENTATION

### 1. Introduction

#### Project Title:

ONLINE COMPLAINT REGISTRATION SYSTEM

#### Team Lead:

| S.No | Team Lead Name     | Role                 |
|------|--------------------|----------------------|
| 1    | Mohammad Arsh Naqi | Full Stack Developer |

#### Team Members:

| S.No | Team Member Name                           | Role                 |
|------|--------------------------------------------|----------------------|
| 1    | Thorlakonda Leela Venkata<br>Koteswara Rao | Full Stack Developer |
| 2    | Nagalakshmi Islavathu                      | Full Stack Developer |
| 3    | Sigullu Chakradhar                         | Full Stack Developer |
| 4    | Thota Pavan Manas                          | Full Stack Developer |

The Online Complaint Registration System is a full-stack web application developed using the MERN stack. The project provides an efficient platform for users to register complaints online and allows administrators and agents to manage, assign, and resolve complaints effectively.

The system aims to improve the traditional complaint handling process by providing a digital solution with better accessibility, tracking, and management features.

## **2. Project Overview**

### **Purpose:**

The purpose of the Online Complaint Registration System is to develop a digital platform that simplifies the process of submitting, managing, and resolving complaints. Traditional complaint handling methods require manual work and consume more time. This system provides an online solution where users can easily register complaints and track their status.

The main goal of this project is to improve communication between users, agents, and administrators by providing a centralized complaint management system. It helps organizations handle complaints efficiently, assign issues to the appropriate agents, and monitor the complete resolution process.

### **Features:**

The Online Complaint Registration System provides the following key features and functionalities:

### **User Module:**

- User registration and secure login.
- Submit complaints through an online platform.
- View previously registered complaints.
- Track complaint status and updates.

- Manage user profile information.

#### **Agent Module:**

- Secure agent login.
- View assigned complaints.
- Analyze complaint details.
- Update complaint progress and status.
- Manage complaint resolution process.

#### **Admin Module:**

- Admin dashboard for complete system management.
- Manage users and agents.
- View all registered complaints.
- Assign complaints to agents.
- Monitor complaint progress and resolution status.

#### **Additional Features:**

- Role-based access control.
- Responsive and user-friendly interface.
- Secure data storage using MongoDB.
- REST API communication between frontend and backend.

### **3. Architecture**

The Online Complaint Registration System follows the MERN stack architecture, which consists of MongoDB, Express.js, React.js, and Node.js. The application is divided into three main layers: Frontend, Backend, and Database. This architecture provides better performance,

scalability, and separation between user interface, business logic, and data management.

### **Frontend Architecture (React.js)**

The frontend of the Online Complaint Registration System is developed using React.js. React provides a component-based architecture that helps create reusable and interactive user interfaces.

The frontend contains different components and pages for Users, Agents, and Administrators. React Router is used for navigation between different pages without reloading the application.

The frontend communicates with the backend server using REST API calls through Axios. User inputs such as login details, registration information, and complaint data are collected from React forms and sent securely to the backend.

Key frontend components include:

- Login and Registration Pages
- User Dashboard
- Complaint Registration Page
- Complaint Tracking Page
- Agent Dashboard
- Admin Dashboard
- Reusable UI Components

### **Backend Architecture (Node.js and Express.js)**

The backend of the application is developed using Node.js and Express.js. It handles business logic, API requests, authentication, and communication between the frontend and database.

Express.js provides RESTful APIs that allow the frontend to perform different operations such as user authentication, complaint creation, complaint assignment, and status updates.

The backend architecture includes:

- Routes: Manage API endpoints and user requests.
- Controllers: Handle application logic and process requests.
- Models: Define database structure and interact with MongoDB.
- Middleware: Handles authentication and request validation.

The backend ensures secure and efficient communication between users, agents, administrators, and the database.

### **Database Architecture (MongoDB)**

MongoDB is used as the database for storing and managing application data. It is a NoSQL database that stores information in flexible document-based collections.

The database contains different collections for managing system information.

Main collections include:

#### **Users Collection:**

Stores user details such as:

- Name
- Email
- Password
- Role information

## **Complaints Collection:**

Stores complaint-related information such as:

- Complaint title
- Description
- Category
- Status
- Created date
- Assigned agent details

## **Agents Collection:**

Stores agent information and handles assigned complaint records.

The backend communicates with MongoDB using Mongoose. User requests from the frontend are processed through Express APIs, stored in MongoDB, and the required response is sent back to the React application.

The complete data flow is:

React Frontend → Express.js API → Node.js Server → MongoDB Database → Response back to User Interface

## **4. Setup Instructions**

### **Prerequisites:**

Before running the Online Complaint Registration System, the following software and tools must be installed:

#### **1. Node.js**

- Required to run the backend server and manage npm packages.
- Provides the runtime environment for executing JavaScript outside the browser.

## **2. MongoDB**

- Used as the database for storing users, agents, and complaint details.
- MongoDB Atlas can also be used for cloud database management.

## **3. Visual Studio Code**

- Used as the code editor for developing and managing the project files.

## **4. npm (Node Package Manager)**

- Used to install and manage frontend and backend dependencies.

## **5. Web Browser**

- Required to access and test the application interface.

### **Installation Steps:**

Follow the below steps to set up the project locally:

#### **Step 1: Clone the Project Repository**

Download or clone the project source code from the repository.

Command:

```
git clone
```

#### **Step 2: Install Frontend Dependencies**

Navigate to the frontend folder:

```
cd client
```

Install required packages:

```
npm install
```

### **Step 3: Install Backend Dependencies**

Navigate to the backend folder:

```
cd server
```

Install required packages:

```
npm install
```

### **Step 4: Configure Environment Variables**

Create a .env file inside the server folder and add the required configuration details.

Example:

```
PORT = 5000
```

```
MONGO_URI = your_mongodb_connection_string
```

```
JWT_SECRET = your_secret_key
```

### **Step 5: Connect Database**

Configure MongoDB connection using the database connection URL.

The backend server connects with MongoDB using Mongoose to perform database operations.

### **Step 6: Start the Application**



Run backend server:

```
npm start
```

Run frontend application:

```
npm start
```

After successful execution, the Online Complaint Registration System will be available in the browser for users, agents, and administrators.

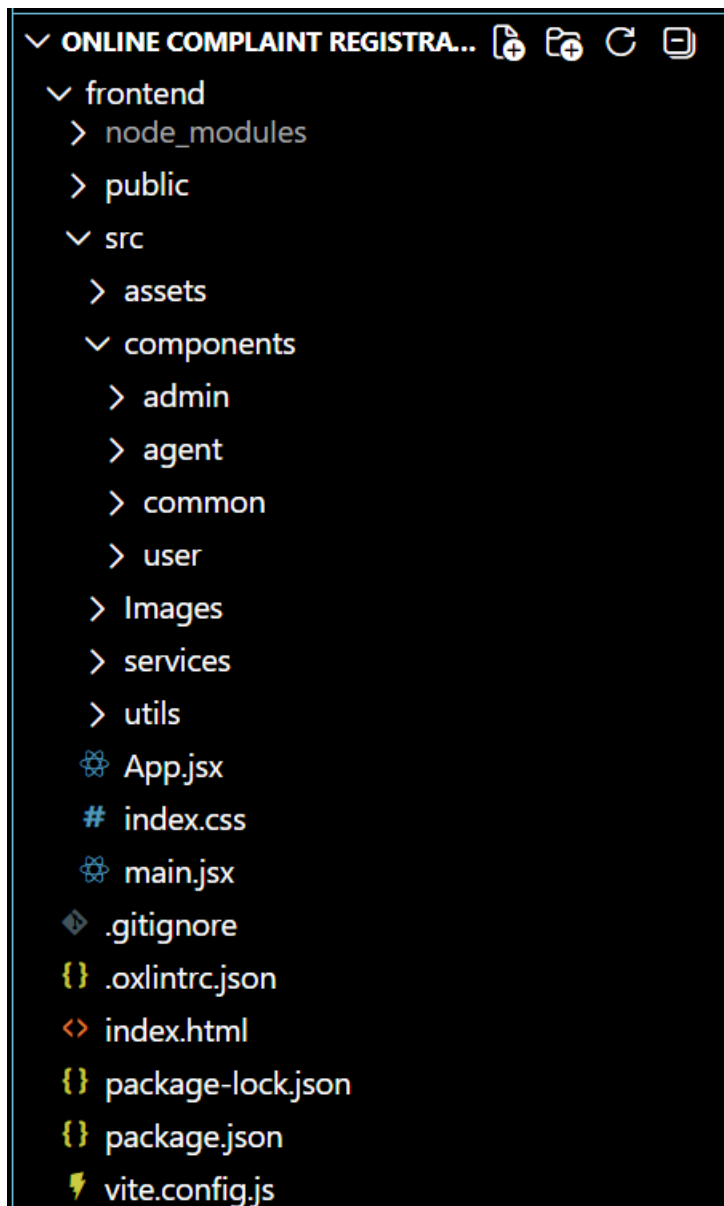
## **5. Folder Structure**

The Online Complaint Registration System follows a structured MERN stack folder organization. The project is divided into two main sections: Client and Server. The Client folder contains the React.js frontend application, while the Server folder contains the Node.js and Express.js backend application.

### **Client (React Frontend)**

The client folder manages the user interface and frontend functionalities of the application. React components, pages, styling files, and API communication services are organized inside this folder.

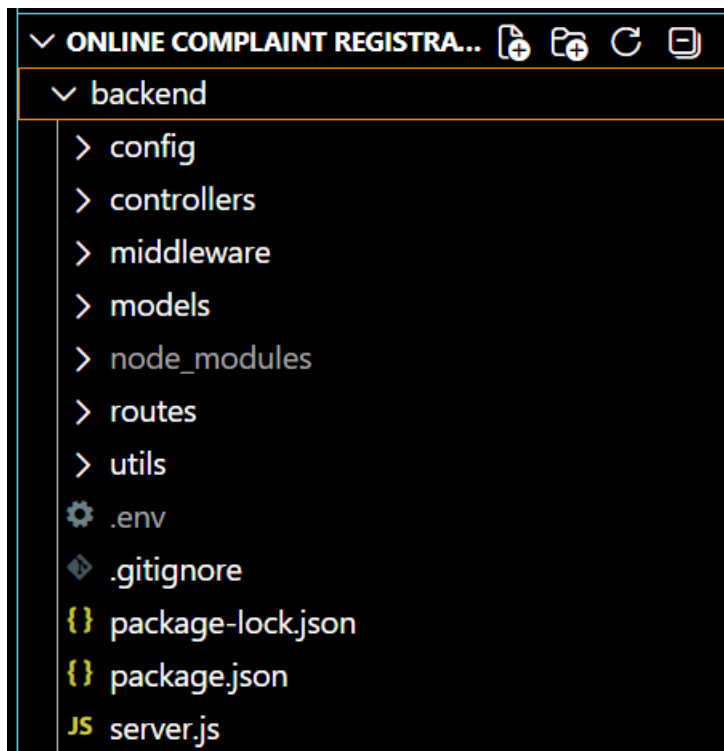
### **Client Folder Structure:**



## Server (Node.js Backend)

The server folder manages backend operations including API creation, database communication, authentication, and business logic.

## Server Folder Structure:



This folder structure improves code organization, readability, and maintainability by separating frontend design, backend logic, and database operations.

## 6. Running the Application

After completing the installation process and configuring all required dependencies, the Online Complaint Registration System can be executed locally by starting both the frontend and backend servers.

### Starting the Frontend Server

The frontend of the application is developed using React.js. To run the frontend application, navigate to the client directory.

Command:

```
cd client
```

Install dependencies (if not installed):

```
npm install
```

Start the React development server:

```
npm start
```

After successful execution, the frontend application will run locally and can be accessed through the browser.

Example:

<http://localhost:5000>

### **Starting the Backend Server**

The backend of the application is developed using Node.js and Express.js. To start the backend server, navigate to the server directory.

Command:

```
cd server
```

Install dependencies (if not installed):

```
npm install
```

Start the backend server:

```
npm start
```

The backend server will start and connect with the MongoDB database.

Example:

<http://localhost:5000>

Once both frontend and backend servers are running successfully, the React frontend communicates with the Express.js backend APIs to perform operations such as authentication, complaint registration, complaint management, and status updates.

## 7. API Documentation

The Online Complaint Registration System uses RESTful APIs developed with Node.js and Express.js. These APIs are responsible for communication between the React frontend and MongoDB database. The APIs handle user authentication, complaint management, agent operations, and admin functionalities.

### Authentication APIs

| Method | Endpoint           | Description                                                 |
|--------|--------------------|-------------------------------------------------------------|
| POST   | /api/auth/register | Registers a new user into the system                        |
| POST   | /api/auth/login    | Authenticates users and provides access based on their role |

### Register API

Request Example:

```
{  
  
"name": "User Name",  
  
"email": "user@gmail.com",
```

```
"password": "password123"
```

```
}
```

Response Example:

```
{
```

```
"message": "User registered successfully",
```

```
"success": true
```

```
}
```

## **Login API**

Request Example:

```
{
```

```
"email": "user@gmail.com",
```

```
"password": "password123"
```

```
}
```

Response Example:

```
{
```

```
"message": "Login successful",
```

```
"token": "authentication_token",
```

```
"role": "user"
```

```
}
```

## Complaint APIs

| Method | Endpoint                   | Description                             |
|--------|----------------------------|-----------------------------------------|
| POST   | /api/complaints/create     | Creates a new complaint                 |
| GET    | /api/complaints            | Retrieves all complaints                |
| GET    | /api/complaints/:id        | Retrieves specific complaint details    |
| PUT    | /api/complaints/update/:id | Updates complaint information or status |

### Create Complaint API

Request Example:

```
{  
  "title": "Internet Issue",  
  "description": "Internet connection is not working",  
  "category": "Network"  
}
```

Response Example:

```
{  
  "message": "Complaint registered successfully",  
  "status": "Pending"  
}
```

## Admin APIs

| Method | Endpoint              | Description                    |
|--------|-----------------------|--------------------------------|
| GET    | /api/admin/users      | Retrieves all registered users |
| GET    | /api/admin/complaints | Displays all user complaints   |
| POST   | /api/admin/agents     | Adds new agent details         |
| PUT    | /api/admin/assign/:id | Assigns complaints to agents   |

## Assign Complaint API

Request Example:

```
{  
  "complaintId": "101",  
  "agentId": "A01"  
}
```

Response Example:

```
{  
  "message": "Complaint assigned successfully"  
}
```

## Agent APIs

| Method | Endpoint              | Description                  |
|--------|-----------------------|------------------------------|
| GET    | /api/agent/complaints | Displays assigned complaints |
| PUT    | /api/agent/status/:id | Updates complaint progress   |



## **Update Complaint Status API**

Request Example:

```
{  
  "status": "Resolved"  
}
```

Response Example:

```
{  
  "message": "Complaint status updated successfully"  
}
```

All API requests and responses are transferred in JSON format. The backend validates requests, performs database operations using MongoDB, and returns appropriate responses to the frontend application.

## **8. Authentication**

Authentication and authorization are important parts of the Online Complaint Registration System to ensure secure access and protect user information. The system implements a secure login mechanism that allows only registered users to access the application features.

### **Authentication Process**

The application uses JSON Web Token (JWT) based authentication for verifying user identity. When a user logs into the system with valid

credentials, the backend verifies the details stored in the MongoDB database.

After successful verification, the server generates a JWT token and sends it back to the frontend. This token is stored on the client side and is used for accessing protected routes and performing authorized operations.

Authentication flow:

1. User enters login credentials.
2. React frontend sends the login request to the Express.js backend API.
3. Backend verifies the user details from MongoDB.
4. After successful verification, a JWT token is generated.
5. The token is sent to the frontend.
6. Future API requests include the token for user verification.

### **Authorization Process**

Authorization is implemented using role-based access control (RBAC). The system contains different roles with different permissions.

#### **User Authorization:**

- Can access user dashboard.
- Can register new complaints.
- Can view and track their complaint status.

#### **Agent Authorization:**

- Can access agent dashboard.
- Can view assigned complaints.
- Can update complaint progress and resolution status.

#### **Admin Authorization:**

- Can access admin dashboard.
- Can manage users and agents.
- Can view all complaints.
- Can assign complaints to agents.

### **Token Security**

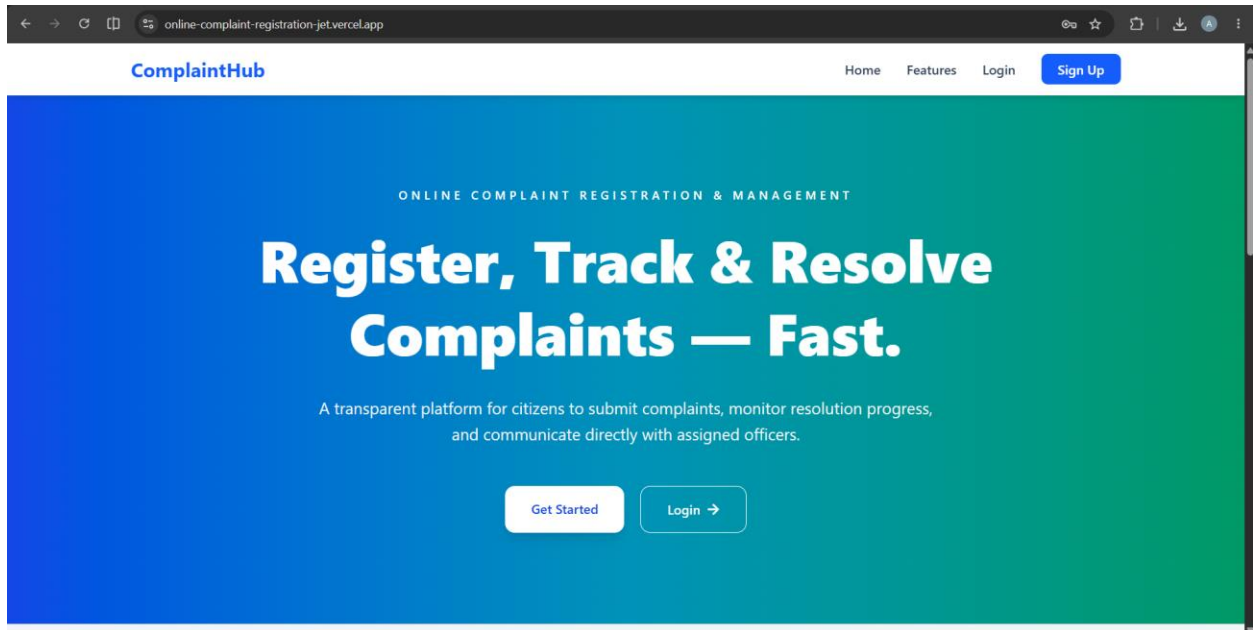
- JWT tokens are used to maintain secure communication between frontend and backend.
- Protected routes verify tokens before allowing access.
- Unauthorized users cannot access restricted dashboards or APIs.

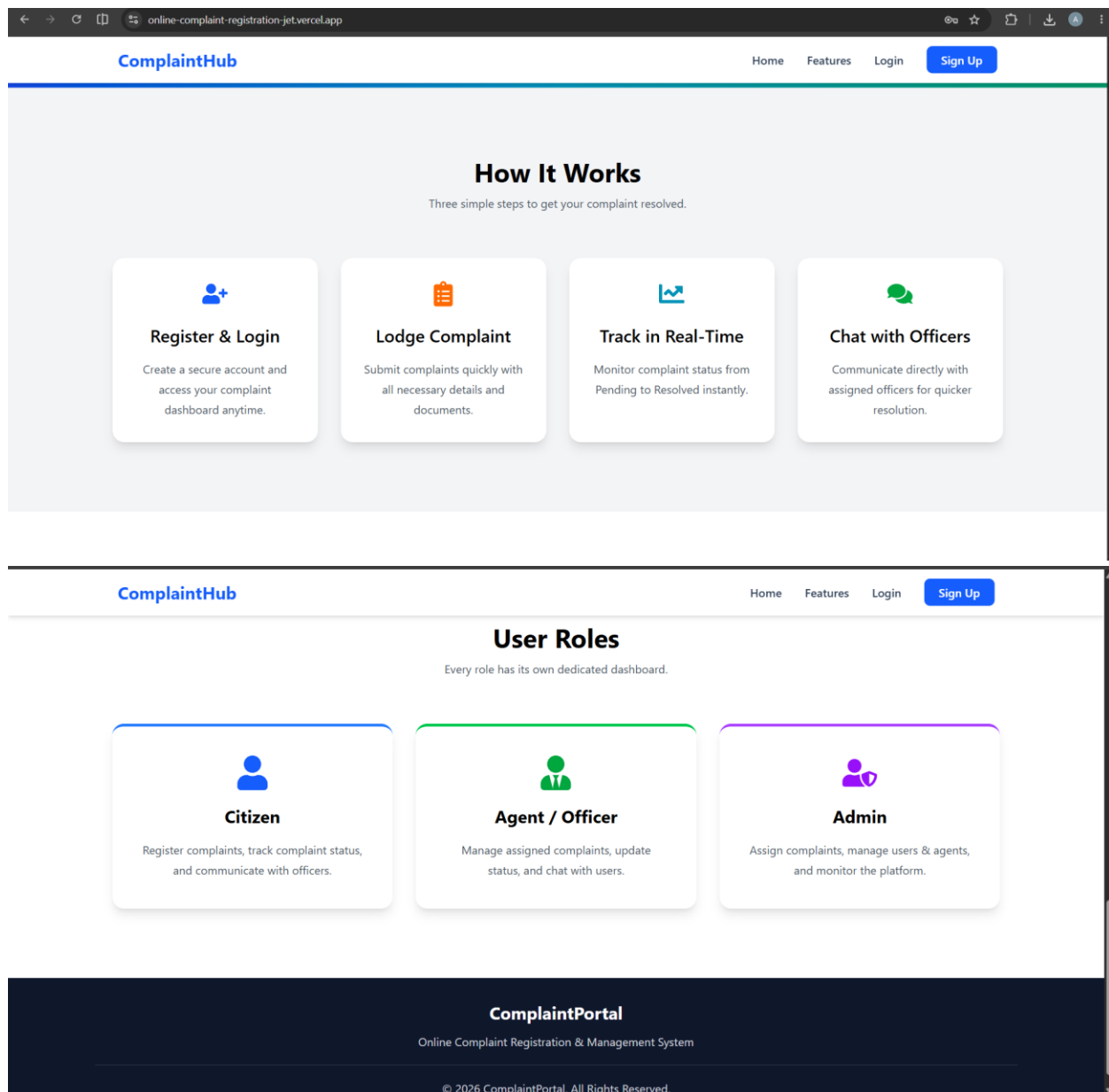
This authentication and authorization mechanism improves application security by ensuring that each user can only access features based on their assigned role.

## **9. User Interface**

The Online Complaint Registration System provides a simple, responsive, and user-friendly interface developed using React.js. The interface is designed separately for Users, Agents, and Administrators to provide easy access to their respective functionalities.

The following screenshots represent different modules and features available in the application:





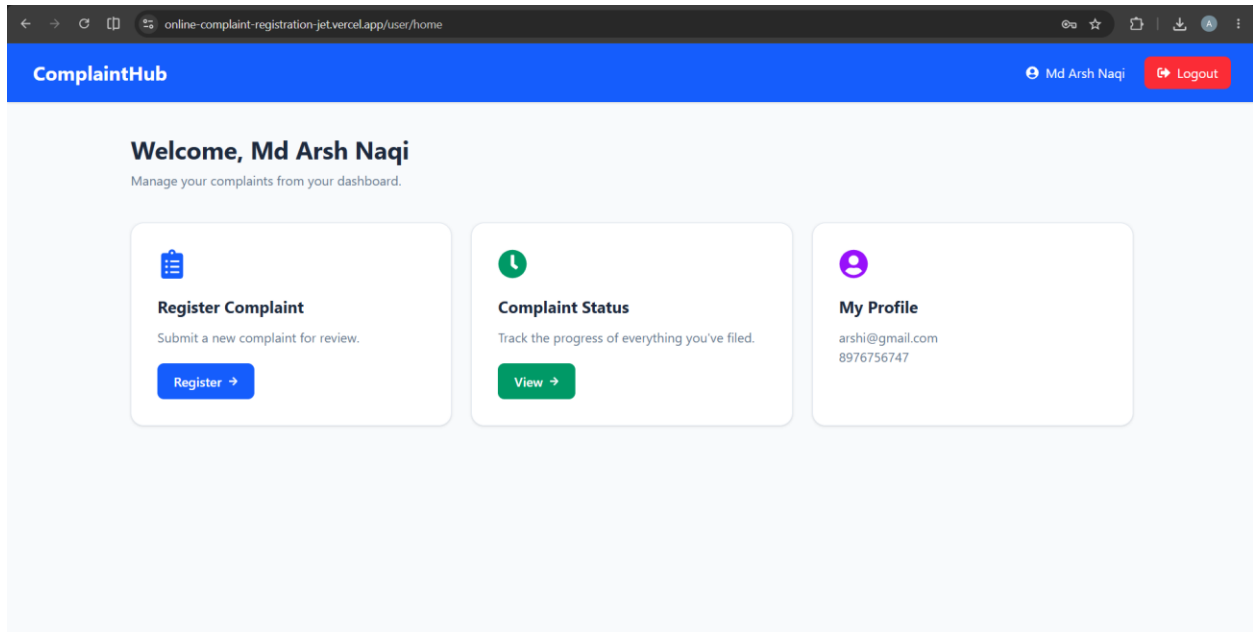
**Figure 1: Home Page of Online Complaint Registration System**

The screenshot shows a web browser window with the URL `online-complaint-registration-jet.vercel.app/signup`. The page has a blue and green gradient background. On the left, a blue box contains the ComplaintHub logo (a shield icon) and the text: "ComplaintHub" and "Create your account and start registering complaints quickly and securely through our complaint portal." On the right, a white box titled "Create Account" with the subtitle "Register as a citizen to start filing complaints" contains four input fields: a name field with "Md Arsh Naqi", an email field with "arshi@gmail.com", a phone number field with "8976756747", and a password field with "\*\*\*\*\*" and a toggle icon. Below these is a blue "Register" button and a link "Already have an account? Login".

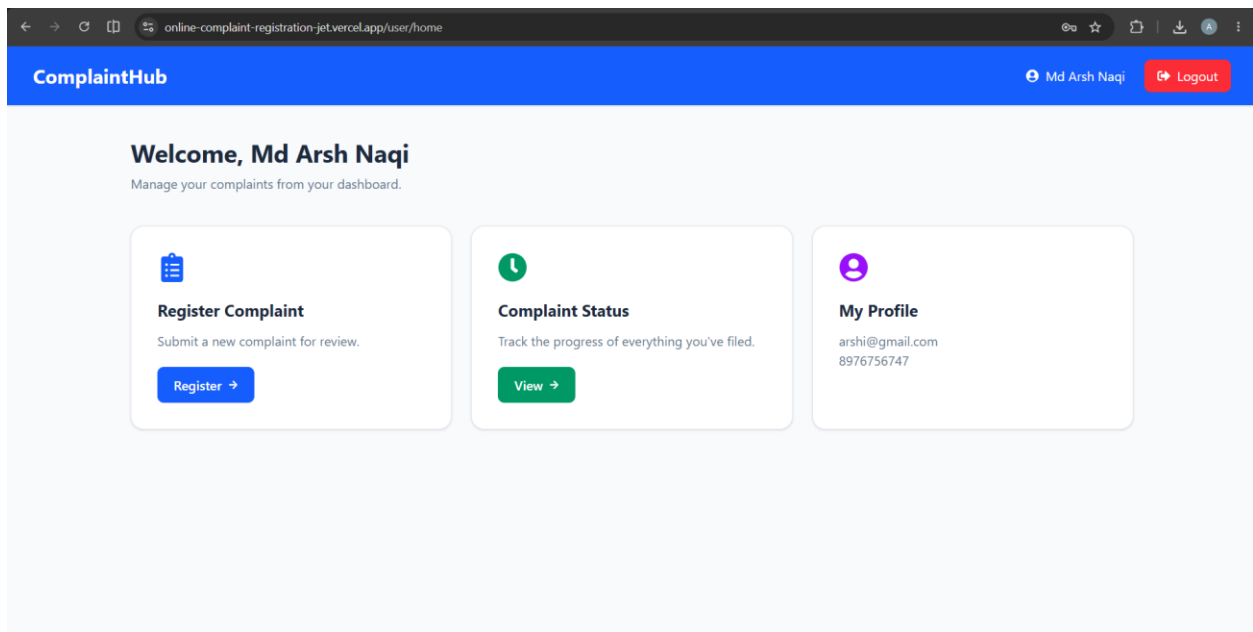
**Figure 2: User Registration Interface**

The screenshot shows a web browser window with the URL `online-complaint-registration-jet.vercel.app/login`. The page has a blue and green gradient background. On the left, a blue box contains the ComplaintHub logo (a shield icon) and the text: "ComplaintHub" and "Register complaints, track status and communicate with officers from one secure platform." On the right, a white box titled "Welcome Back" with the subtitle "Login to your account" contains two input fields: an email field with "arshi@gmail.com" and a password field with "\*\*\*\*\*" and a toggle icon. Below these is a blue "Login" button and a link "Don't have an account? Register".

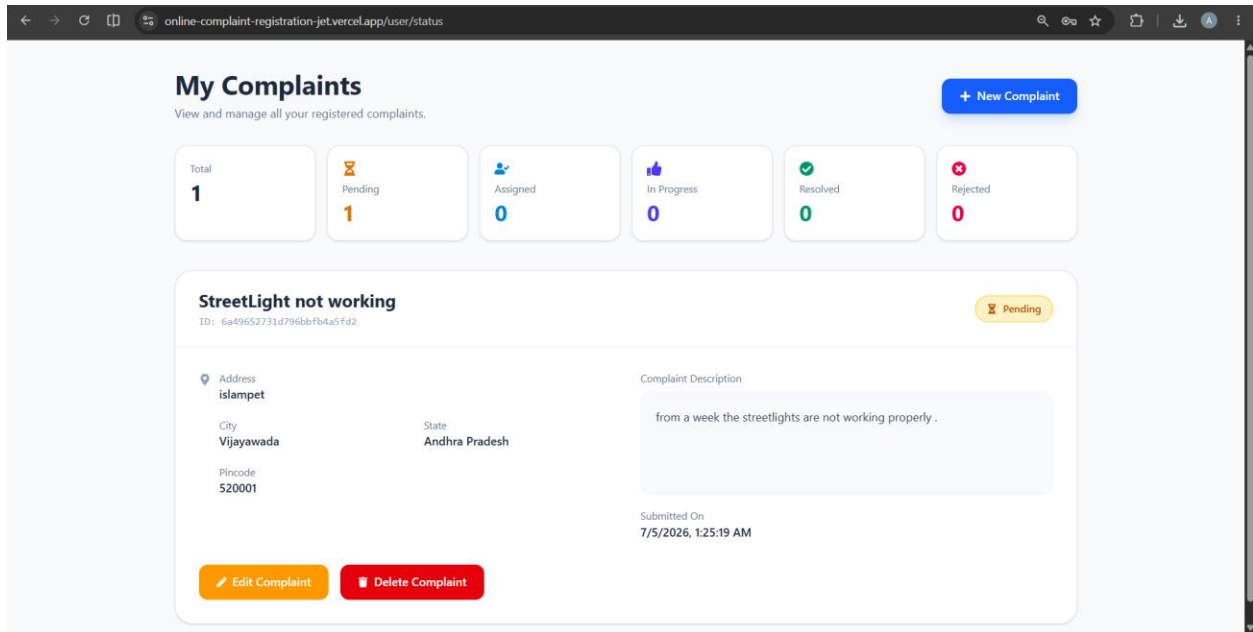
**Figure 3: Login Interface**



**Figure 4: User Dashboard**

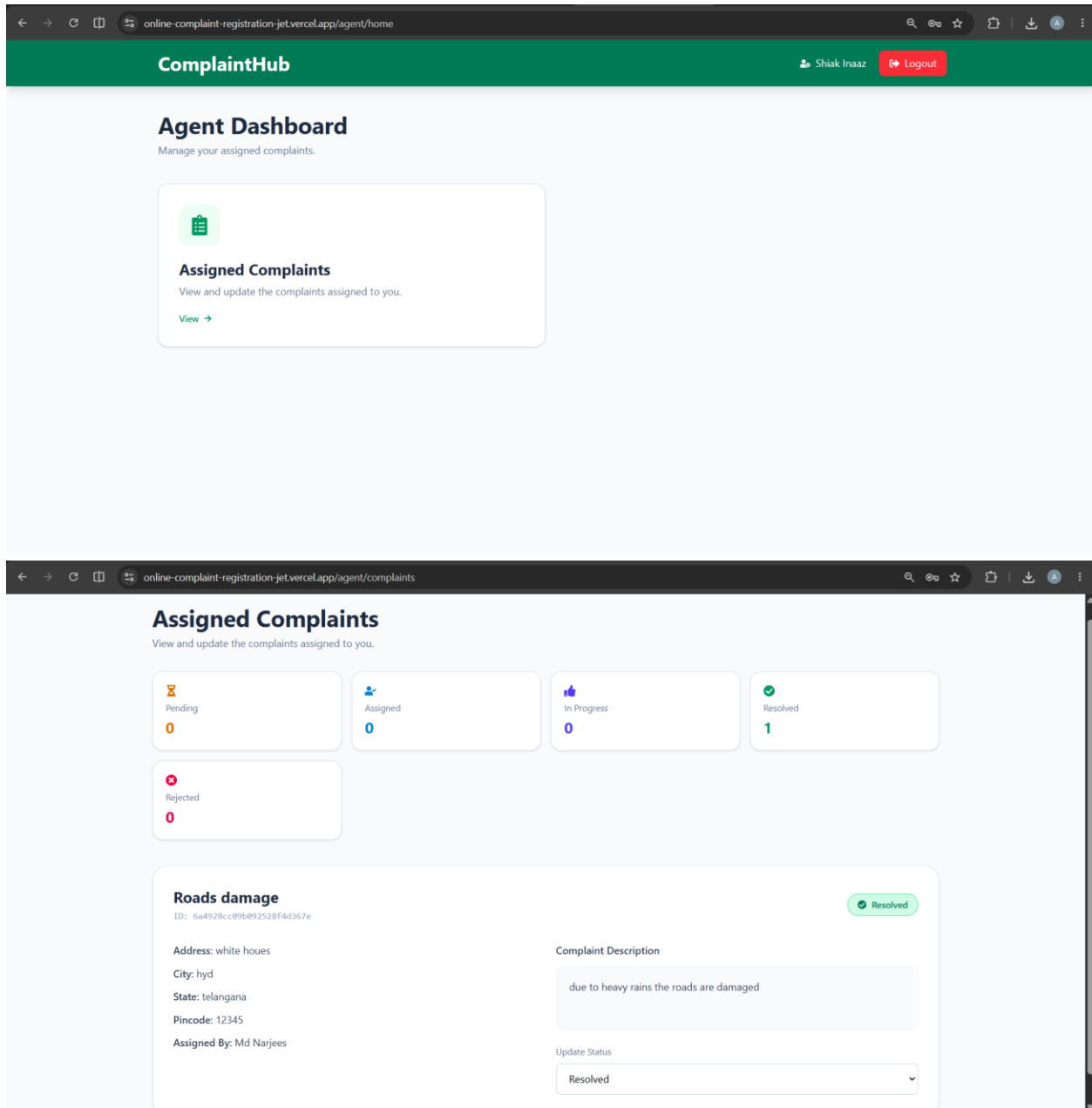


**Figure 5: Complaint Registration Interface**



**Figure 6: Complaint Tracking Interface**





**Figure 7: Agent Dashboard**

Admin Dashboard

Manage users, agents and complaints across the platform.



**Users**  
View all registered citizens.  
[View →](#)



**Agents**  
View all complaint-handling agents.  
[View →](#)



**Complaints**  
Review and assign complaints to agents.  
[View →](#)

 Registered Users

View all users registered in ComplaintHub.

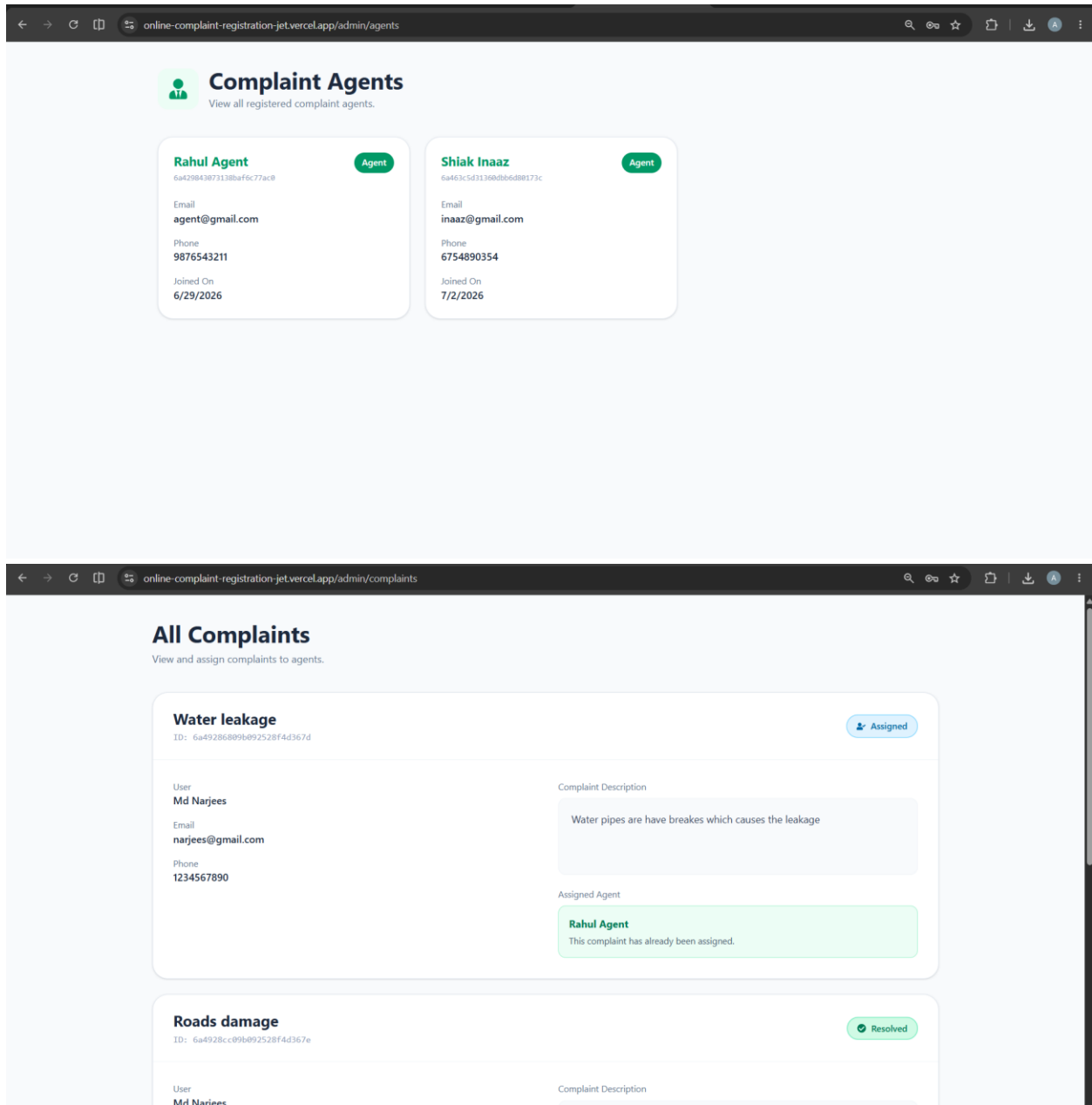
**Rahul Agent** Agent  
6a429843873138ba6c77ac8  
Email  
agent@gmail.com  
Phone  
9876543211  
Joined On  
6/29/2026

**Md Narjees** User  
6a455faf3be7941dae11db3b  
Email  
narjees@gmail.com  
Phone  
1234567890  
Joined On  
7/2/2026

**Shiak Inaaz** Agent  
6a463c5d31360dbb6d88173c  
Email  
inaaz@gmail.com  
Phone  
6754890354  
Joined On  
7/2/2026

**Shaik Ishaan** Admin  
6a463d9831360dbb6d88173f  
Email  
ishaan@gmail.com  
Phone  
9856346210  
Joined On  
7/2/2026

**Md Arsh Naqi** User  
6a496a7a31e796bbfb4a5fd1  
Email  
arshi@gmail.com  
Phone  
8976756747  
Joined On  
7/5/2026



**Figure 8: Admin Dashboard**

## 10. Testing

Testing is an important phase in the development of the Online Complaint Registration System. It ensures that all modules of the application work

correctly, securely, and efficiently. Different testing methods were performed to verify frontend functionality, backend APIs, authentication, and database operations.

## **Testing Strategy**

The application was tested using different testing approaches:

### **1. Frontend Testing**

Frontend testing was performed to verify the functionality and user interface of the React.js application.

The following features were tested:

- User registration form validation.
- Login functionality.
- Complaint submission forms.
- Navigation between different pages.
- Dashboard components for User, Agent, and Admin.
- Responsive design on different screen sizes.

### **2. Backend API Testing**

Backend APIs developed using Node.js and Express.js were tested to ensure proper request handling and response generation.

API testing included:

- Checking API endpoints.
- Verifying request and response data.
- Testing CRUD operations.
- Validating error handling.

Tool Used:

Postman was used to test REST API endpoints and verify backend functionality.

### 3. Authentication Testing

Authentication testing was performed to verify secure access control.

Test cases included:

- Valid user login.
- Invalid login attempts.
- Role-based dashboard access.
- Protected route verification.

### 4. Database Testing

MongoDB database testing was performed to check data storage and retrieval operations.

Database testing included:

- User data insertion.
- Complaint record creation.
- Data updates.
- Retrieving complaint information.

### Testing Tools Used

| Tool                        | Purpose                      |
|-----------------------------|------------------------------|
| Postman                     | API testing                  |
| Browser Developer Tools     | Debugging frontend issues    |
| MongoDB Atlas               | Database verification        |
| Visual Studio Code Terminal | Error checking and debugging |

After successful testing, all major functionalities including user management, complaint registration, complaint assignment, and status updates were verified successfully.

## **11. Screenshots or Demo**

The Online Complaint Registration System has been successfully developed and deployed. The live demo allows users to access and test the application features including user registration, complaint submission, complaint tracking, agent management, and admin operations.

### **Live Demo Link:**

<https://online-complaint-registration-jet.vercel.app/>

The deployed application demonstrates the complete workflow of the complaint management system with separate modules for Users, Agents, and Administrators.

The major functionalities available in the demo include:

- User account creation and authentication.
- Online complaint registration.
- Complaint status tracking.
- Agent complaint handling.
- Admin monitoring and management.

The live application provides a user-friendly interface and shows the integration between React.js frontend, Node.js/Express.js backend, and MongoDB database.

## **12. Known Issues**

The Online Complaint Registration System has been developed and tested successfully. The major functionalities of the application are working as expected. However, there are some areas that can be improved in future versions.

The following are the known issues and limitations:

### **1. Notification System**

- Currently, users need to manually check their complaint status.
- Real-time email or SMS notifications are not implemented.

### **2. Real-Time Updates**

- Complaint status changes require page refresh or manual checking.
- Real-time updates using WebSocket technology can be added.

### **3. Advanced Search and Filtering**

- Basic complaint viewing functionality is available.
- Advanced filtering based on priority, category, and date can be improved.

### **4. File Attachment Support**

- Users can submit complaint details.
- Uploading supporting documents or images for complaints can be added.

### **5. Mobile Application Availability**

- The system is currently available as a web application.

- A dedicated mobile application can be developed for better accessibility.

## **6. AI-Based Complaint Analysis**

- The current system manages complaints manually.
- AI-based complaint classification and priority detection can be implemented in future versions.

These limitations do not affect the main functionality of the system and can be considered as future improvements to enhance performance and user experience.

## **13. Future Enhancements**

The Online Complaint Registration System provides the essential features required for complaint registration and management. In future versions, additional features and improvements can be implemented to make the application more advanced, efficient, and user-friendly.

The following enhancements can be added:

### **1. Real-Time Notifications**

A notification system can be implemented to provide instant updates to users about their complaint status. Email and SMS alerts can notify users whenever their complaint is assigned, updated, or resolved.

### **2. Live Chat Support**

A real-time chat feature can be introduced to improve communication between users and support agents. This will help users receive quick responses regarding their complaints.



### **3. AI-Based Complaint Classification**

Artificial Intelligence can be integrated to automatically analyze complaints and categorize them based on type, priority, and urgency. This helps in faster complaint assignment and resolution.

### **4. File Upload Feature**

Users can be provided with an option to upload images, documents, or other proof related to their complaints. This will help agents understand and resolve issues more effectively.

### **5. Advanced Analytics Dashboard**

The admin dashboard can be improved by adding graphical reports and analytics to monitor:

- Total complaints
- Resolved complaints
- Pending issues
- Agent performance

### **6. Mobile Application Development**

A dedicated mobile application can be developed for Android and iOS platforms to provide easier access and better user experience.

### **7. Improved Search and Filtering**

Advanced search and filtering options can be added to quickly find complaints based on category, date, status, and priority level.

## **8. Multi-Language Support**

The application can support multiple languages, allowing users from different regions to access and use the system comfortably.

These future enhancements will increase the efficiency, accessibility, and overall performance of the Online Complaint Registration System.